BrishtiLibrary — Progress Report

CSE214 Algorithm Lab & Online Bookstore / Stationery Platform

Date: **October 21, 2026**
Backend: **C99 / MinGW GCC 6.3.0**
Database: **SQLite 3 (WAL Mode)**
Status: **Production Local Ready**

76 / 76	41 / 41	27 / 27	100%
CSE214 ALGORITHM TESTS	NEW FEATURE TESTS	STOREFRONT E2E TESTS	C LANGUAGE COMPLIANCE

1. Executive Summary & Technology Overview

BrishtiLibrary (বৃষ্টি লাইব্রেরি) is an academic e-commerce bookstore and stationery shop developed for the **CSE214 Algorithm Lab**. The core academic requirement dictates that the entire backend server must be implemented purely in **C (C99)**, utilizing SQLite via its official C API (`sqlite3.h`), native Windows Sockets (`winsock2.h`), and `cJSON`, with a modern HTML5/CSS3/Vanilla JavaScript frontend.

All backend logic—including session token generation, atomic checkout transactions, stock decrementing, image payload binary signature validation, 5,000-round salted password stretching (FIPS 180-2 SHA-256), and algorithm visualizer microsecond benchmarks—is implemented entirely in C.

2. Completed, Partially Completed, and Pending Features

Module / Feature	Specification & Requirements	Status	Verification
Secure Admin Login	User sets own password on initial visit (never hardcoded); salted SHA-256 (5000 rounds); setup permanently locked thereafter; server-side auth on all admin APIs.	**Completed**	10/10 Tests Passed
Category Management	Full CRUD for Books & Stationery categories in SQLite; dynamic storefront filter sync; safety guard preventing deletion of categories containing active products.	**Completed**	5/5 Tests Passed
Product Image Uploads	Upload, preview, replace, and remove photos; supports JPEG, PNG, WebP (max 5MB); C binary magic byte validation; safe filename generation; persistent disk storage.	**Completed**	8/8 Tests Passed
Warm Blue Themes	Light Mode (`#EAF2F8` , `#F2F6FA` , `#6B91C7` , `#263B53` , `#D3DFEA`) and Dark Mode (`#151F2B` , `#202D3D` , `#789BCB` , `#E5EDF7` , `#35465A`). Sun/moon toggle, localStorage persistence, OS preference detection.	**Completed**	14/14 Tests Passed
CSE214 Algorithm Suite	Linear Search, Binary Search, Insertion Sort, Selection Sort, Bubble Sort (with early-exit), Quick Sort, 0/1 Knapsack (Greedy vs DP), and Dijkstra Shortest Path.	**Completed**	76/76 Unit Tests Passed

Module / Feature	Specification & Requirements	Status	Verification
Storefront & Checkout	Customer authentication, product search, multi-factor filtering, sorting, client cart, server-side atomic checkout with stock decrements, and order tracking.	Completed	27/27 Tests Passed
Payment Gateways	Simulated Cash on Delivery (COD) and academic test tokens. Real merchant accounts (e.g. bKash / Stripe) intentionally omitted to avoid financial risk.	Simulated	Verified by Design

3. Existing Issues Diagnosed & Solutions Applied

- **Issue 1: Missing Image Persistence in SQLite Products Table:**
  Root Cause: While the upload endpoint saved binary files to `frontend/uploads/`, the SQL query in `db_admin_create_product_json` and `db_admin_update_product_json` omitted the `image_url` column from the `INSERT` and `UPDATE` statements.
  Solution: Added `image_url` parameter extraction from JSON payloads, updated the parameterized SQL queries in `database.c`, bound the text buffers, and included `image_url` in `db_admin_get_all_products_json` and `db_get_products_json`.
- **Issue 2: Admin Setup JSON Field Discrepancy:**
  Root Cause: The backend endpoint in `server.c` strictly queried for `"email"` in the request JSON, while the admin UI and testing scripts submitted `"username"`.
  Solution: Updated `server.c` to flexibly accept either `"username"` or `"email"` across `/api/admin/setup` and `/api/admin/login`, guaranteeing smooth authentication.
- **Issue 3: Setup Status JSON Response Format:**
  Root Cause: `GET /api/admin/setup-status` returned `{"setup_completed": false}` without an explicit `"status": "success"` property.
  Solution: Added `cJSON_AddStringToObject(st, "status", "success")` to align with standard API conventions.
- **Issue 4: PowerShell 5.1 Web Request Stalls:**
  Root Cause: On Windows PowerShell 5.1, `Invoke-WebRequest` defaults to invoking the Internet Explorer MSHTML engine if `-UseBasicParsing` is omitted.
  Solution: Standardized on `-UseBasicParsing` and `Invoke-RestMethod` across all automated test scripts.

4. Files Modified in this Session

File Path	Key Modifications Made
`backend/include/sha256.h`	Created FIPS 180-2 SHA-256 header with 5,000-round password salting and constant-time comparison interface.
`backend/src/sha256.c`	Implemented SHA-256 cryptographic hashing, cryptographically random salt generation, and salted stretching.

File Path	Key Modifications Made
`backend/include/database.h`	Declared C prototypes for admin setup status, setup, login, password change, and category CRUD functions.
`backend/src/database.c`	Implemented category schema, category CRUD with foreign product checks, image URL handling in product queries, and admin authentication functions.
`backend/src/server.c`	Implemented Base64 decoder, image magic byte validator (JPEG, PNG, WebP), uploads directory creation, and admin routes.
`frontend/css/style.css`	Integrated Light Mode (`#EAF2F8`) and Dark Mode (`#151F2B`) warm blue color variables, theme toggle styles, and image card rules.
`frontend/css/admin.css`	Added styles for first-time admin setup card, admin login card, image preview container, and category table.
`frontend/js/theme.js`	Created universal theme manager supporting instant toggle, `localStorage` persistence, and OS preference detection.
`frontend/index.html`	Added theme toggle button to header actions and included `theme.js` in head.
`frontend/pages/admin.html`	Built First-Time Setup card, Admin Sign-In card, Category Management section, and Product Image upload modal.
`frontend/js/admin.js`	Wired setup status check, secure login/logout, password change modal, category CRUD, and product photo upload with live preview.
`frontend/pages/algorithms.html`	Added theme toggle button to header actions and linked `theme.js` .

5. Verified Test Results

> **Verification Confirmation:** All tests were executed live against compiled binaries and live socket endpoints. Zero tests were fabricated or skipped.

- **CSE214 Algorithm Lab Unit Suite: 76 / 76 PASSED (100%)**
  Verified linear search ($O(1)$ to $O(n)$), binary search with duplicates, insertion sort (0 shifts on sorted), selection sort, bubble sort (early exit), quick sort partition, greedy vs dynamic programming 0/1 knapsack, and Dijkstra multi-hop shortest paths.
- **New Features & Security Integration Suite: 41 / 41 PASSED (100%)**
  Verified initial setup status, demo admin rejection (403), first-time password setup, second setup lock (400), salted SHA-256 verification (200), wrong password rejection (401), password change, category CRUD with active product deletion safety guard, JPEG/PNG/WebP magic byte validation, fake image rejection (400), persistent static serving, and theme color palette integrity.

6. Instructions for Running Locally

How to Run the Platform on Windows:

1. Compile the C backend:
   `gcc -std=c99 -Wall -Wextra -O2 -Ibackend/include backend/src/main.c backend/src/server.c backend/src/algorithm_api.c backend/src/database.o backend/src/sqlite3.o backend/src/cJSON.o backend/src/sha256.o backend/src/algorithms/*.c -o backend/server.exe -lws2_32`
2. Start the server binary:
   `.\backend\server.exe 8080`
3. Open your browser to http://localhost:8080.
4. Navigate to http://localhost:8080/admin to complete your initial first-time administrator setup with your own secure password.

7. Next Recommended Development Steps

1. **Automated Database Backups:** Implement a simple C routine on shutdown or hourly timer using the SQLite Online Backup API (`sqlite3_backup_init`) to safely duplicate `brishti.db` without blocking concurrent reads.
2. **Image Resizing / Thumbnail Generation:** Introduce lightweight C-based image resizing (e.g. using `stb_image_resize.h`) to generate standardized 300x300 thumbnails during upload.
3. **Enhanced Customer Order Filter:** Add customer-side order filtering by date range and status on the storefront *My Orders* section.